Exercise 10 — Define and Call Methods

Unit 3 — Working with Local Classes

Global Class: ZCL_9309_LOCAL_CLASS

Solution concept: Define and implement methods in local class LCL_CONNECTION, call them from IF_OO_ADT_CLASSRUN~MAIN, and handle CX_ABAP_INVALID_VALUE.

Objective

In this exercise, the local class LCL_CONNECTION is extended with two instance methods: SET_ATTRIBUTES and GET_OUTPUT. The methods are implemented and then called from the main method. The exercise also demonstrates exception handling with TRY...CATCH.

Task 1 — Copy Template (Optional)

The exercise can continue from the previous Create and Manage Instances exercise. Because ZCL_9309_LOCAL_CLASS was already created and contains LCL_CONNECTION, copying the template is optional.

If starting from the SAP template:

1. 1. Open /LRN/CL_S4D400_CLS_INSTANCES.
2. 2. Link the Project Explorer with the editor.
3. 3. Duplicate it into your package as ZCL_##_METHODS.

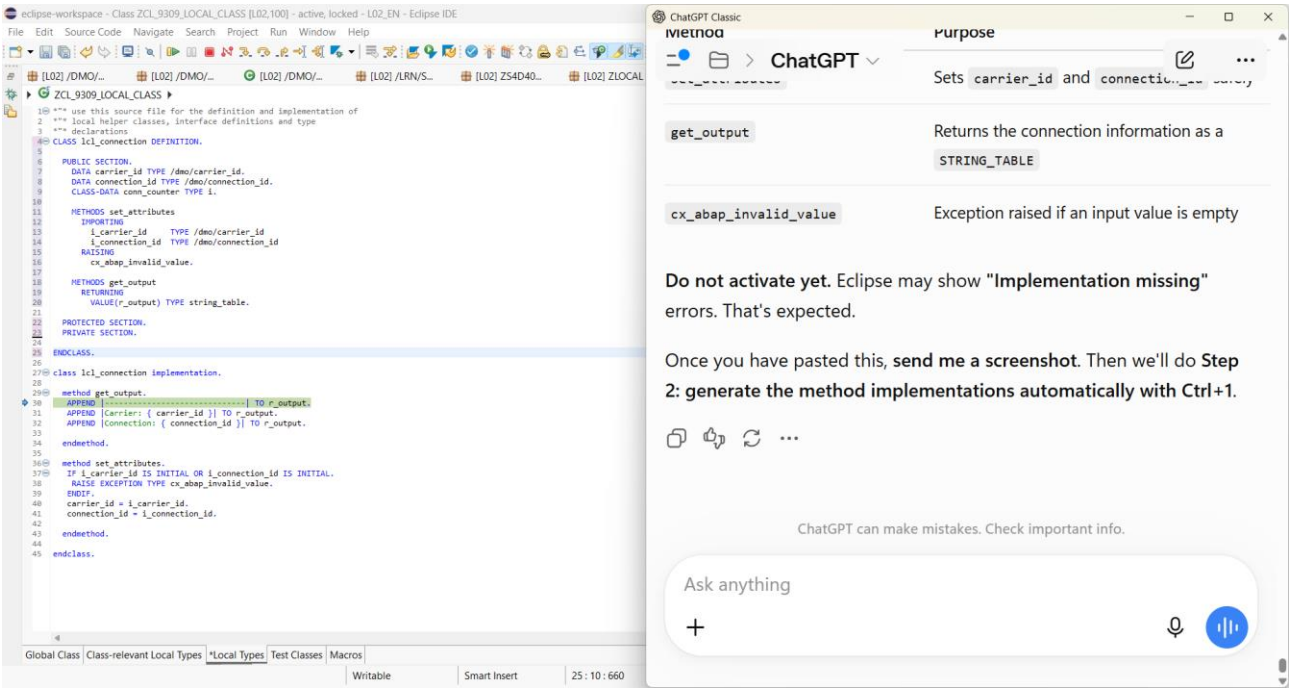
Task 2 — Define Methods

Open the Local Types tab and add the following method declarations inside the PUBLIC SECTION of LCL_CONNECTION:

```
METHODS set_attributes
  IMPORTING
    i_carrier_id    TYPE /dmo/carrier_id
    i_connection_id TYPE /dmo/connection_id
  RAISING
    cx_abap_invalid_value.

METHODS get_output
  RETURNING
    VALUE(r_output) TYPE string_table.
```

Purpose:

- SET_ATTRIBUTES receives carrier and connection values and validates them.
- GET_OUTPUT returns formatted connection information as STRING_TABLE.
- CX_ABAP_INVALID_VALUE is raised when either input is initial.



Screenshot 1 — Local Types with SET_ATTRIBUTES and GET_OUTPUT implementations.

Task 3 — Implement Methods

Use Ctrl+1 on an unimplemented method and choose “Add 2 unimplemented methods”. Then implement the methods.

GET_OUTPUT implementation:

```
METHOD get_output.
  APPEND |------------------------------| TO r_output.
  APPEND |Carrier: { carrier_id }| TO r_output.
  APPEND |Connection: { connection_id }| TO r_output.
ENDMETHOD.
```

SET_ATTRIBUTES implementation:

```
METHOD set_attributes.
  IF i_carrier_id IS INITIAL OR i_connection_id IS INITIAL.
    RAISE EXCEPTION TYPE cx_abap_invalid_value.
  ENDIF.

  carrier_id = i_carrier_id.
  connection_id = i_connection_id.
ENDMETHOD.
```

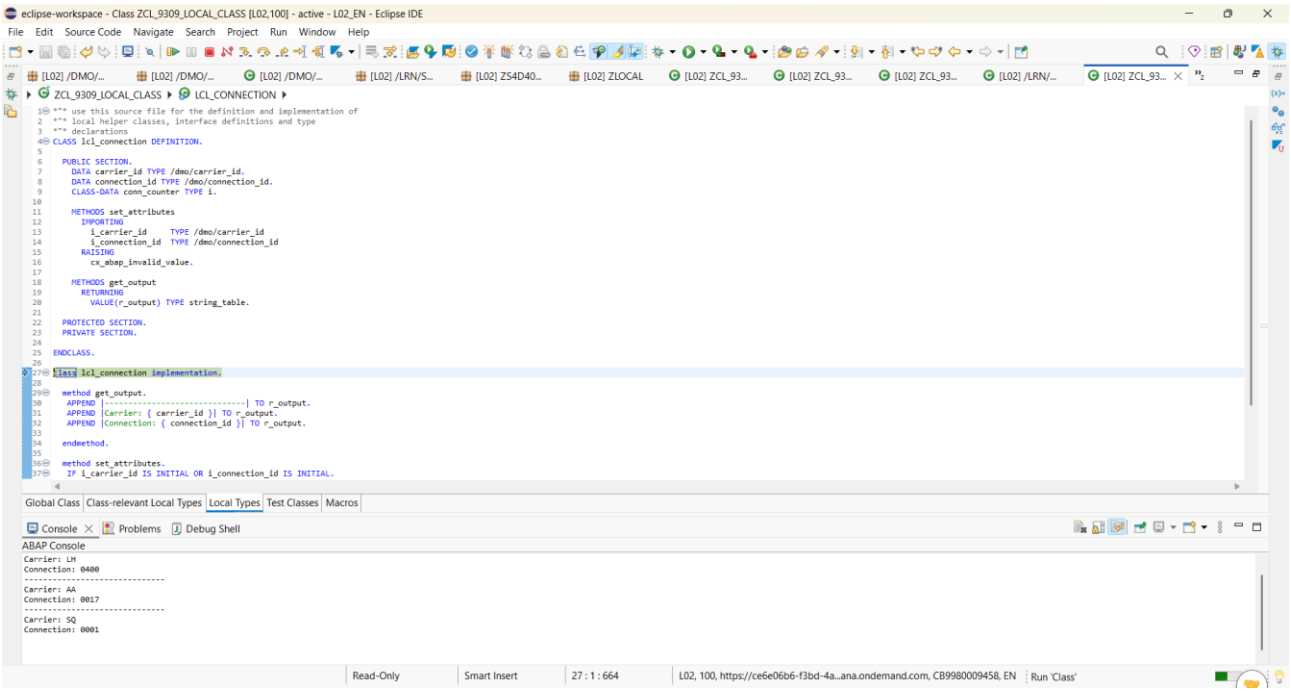
Activate with Ctrl+F3. The class must activate without errors.

Task 4 — Call the Functional Method GET_OUTPUT

At the end of IF_OO_ADT_CLASSRUN~MAIN, loop through the internal table and call GET_OUTPUT:

```
LOOP AT connections INTO connection.
  out->write( connection->get_output( ) ).
ENDLOOP.
```

Run the class with F9. The console should display the three connection objects.



Screenshot 2 — Successful console output for the three connections.

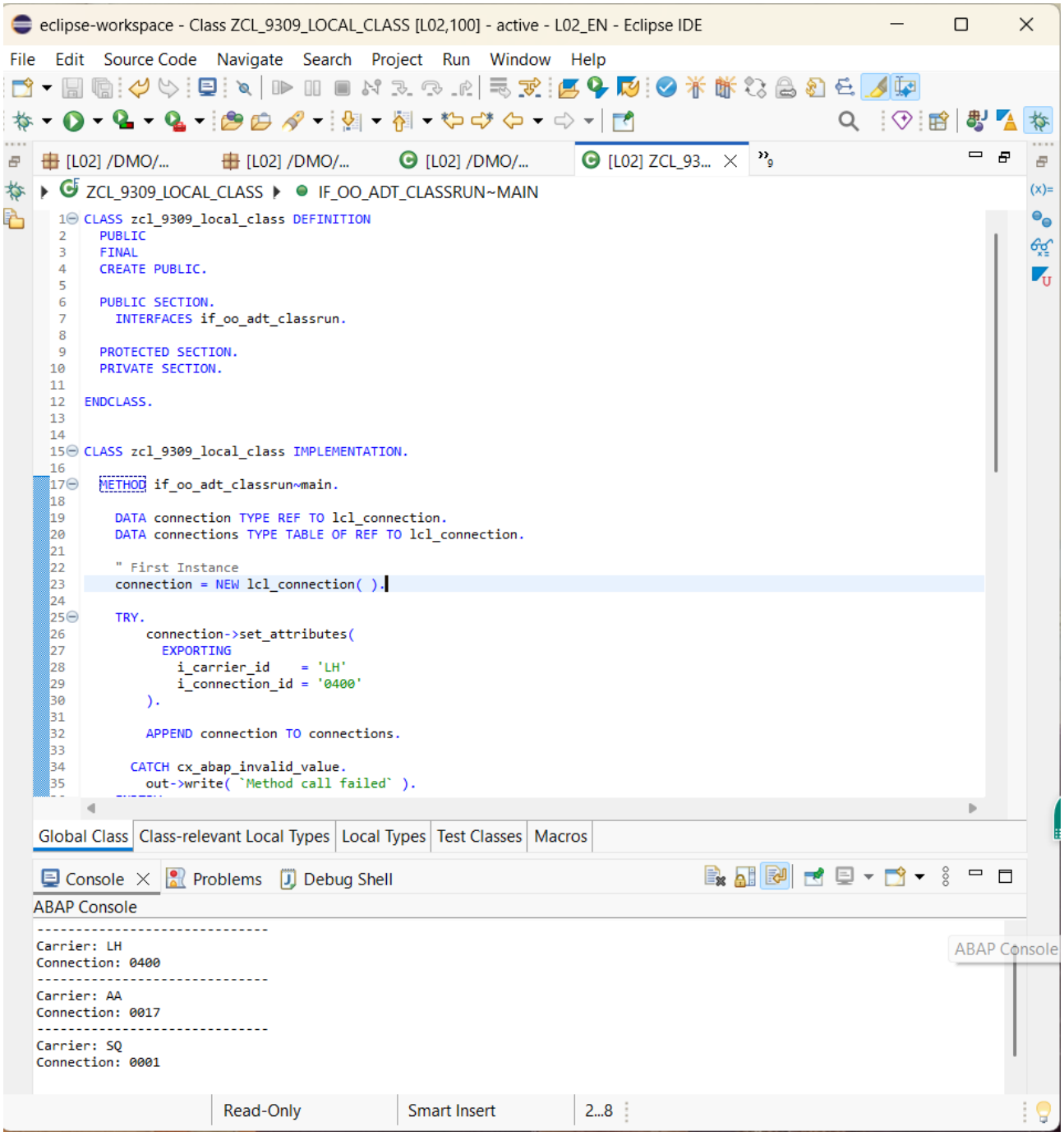
Task 5 — SET_ATTRIBUTES and Exception Handling

Replace direct assignments to CARRIER_ID and CONNECTION_ID with SET_ATTRIBUTES. Because SET_ATTRIBUTES raises CX_ABAP_INVALID_VALUE, surround the method call with TRY...CATCH. Only append the object to CONNECTIONS after the method call succeeds.

Complete IF_OO_ADT_CLASSRUN~MAIN

```
METHOD if_oo_adt_classrun~main.

  DATA connection TYPE REF TO lcl_connection.
  DATA connections TYPE TABLE OF REF TO lcl_connection.

  " First Instance
  connection = NEW lcl_connection( ).

  TRY.
      connection->set_attributes(
        EXPORTING
          i_carrier_id    = 'LH'
          i_connection_id = '0400'
      ).

      APPEND connection TO connections.

    CATCH cx_abap_invalid_value.
      out->write( `Method call failed` ).
  ENDTRY.

  " Second Instance
  connection = NEW lcl_connection( ).

  TRY.
```

```
      connection->set_attributes(
        EXPORTING
          i_carrier_id    = 'AA'
          i_connection_id = '0017'
      ).

      APPEND connection TO connections.

    CATCH cx_abap_invalid_value.
      out->write( `Method call failed` ).
  ENDTRY.

  " Third Instance
  connection = NEW lcl_connection( ).

  TRY.
      connection->set_attributes(
        EXPORTING
          i_carrier_id    = 'SQ'
          i_connection_id = '0001'
      ).

      APPEND connection TO connections.

    CATCH cx_abap_invalid_value.
      out->write( `Method call failed` ).
  ENDTRY.

  " Output
  LOOP AT connections INTO connection.
    out->write( connection->get_output( ) ).
  ENDLOOP.

ENDMETHOD.
```



Screenshot 3 — Final Exercise 10 code with SET_ATTRIBUTES, TRY...CATCH, and correct console output.

What the Code Demonstrates

Concept	Meaning
Instance method	A method that operates on one object instance.
IMPORTING parameter	Passes input values into SET_ATTRIBUTES.
RETURNING parameter	GET_OUTPUT returns a STRING_TABLE.
NEW	Creates a new LCL_CONNECTION object.
->	Accesses an instance method or attribute through an object reference.
TRY...CATCH	Handles the CX_ABAP_INVALID_VALUE exception.
APPEND	Stores the object reference in the CONNECTIONS internal table.

Functional method	GET_OUTPUT can be used directly as an argument to OUT->WRITE.

Expected Console Output

```
------------------------------
Carrier: LH
Connection: 0400
------------------------------
Carrier: AA
Connection: 0017
------------------------------
Carrier: SQ
Connection: 0001
```

Completion Checklist

☐ LCL_CONNECTION contains SET_ATTRIBUTES.

☐ LCL_CONNECTION contains GET_OUTPUT.

☐ Both methods have implementations.

☐ SET_ATTRIBUTES validates initial values.

☐ CX_ABAP_INVALID_VALUE is handled with TRY...CATCH.

☐ All three instances use SET_ATTRIBUTES.

☐ All successful objects are appended to CONNECTIONS.

☐ GET_OUTPUT is called inside the LOOP.

☐ Class activates successfully.

☐ F9 execution produces the expected three connections.

Final Status

EXERCISE 10 — COMPLETED

The final implementation successfully demonstrates defining local class methods, implementing them, calling a functional method, passing importing parameters, creating instances, storing references in an internal table, and handling an ABAP exception.